Informe de Laboratorio 4

Michelle Sayas. C.I: 31.717.191
y Tulio Cordero. C.I: 30571603

Octubre 2026

1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).

El ciclo de una interrupción de hardware comienza cuando un dispositivo activa su línea de petición, y termina cuando el procesador reanuda el programa interrumpido. La Figura 1 resume las etapas.

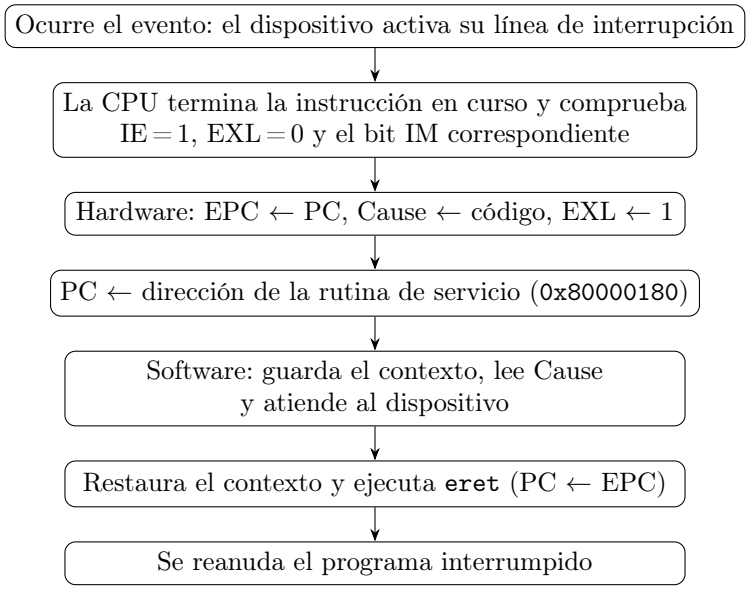


Figura 1: Ciclo de una interrupción de hardware.

En los ejercicios de esta práctica el evento es la pulsación de una tecla en el *Keyboard and Display MMIO Simulator*: el dispositivo deja el carácter en `0xFFFF0004`, activa el bit *Ready* y, como el bit de habilitación de `0xFFFF0000` está en 1, levanta la interrupción. La rutina situada en `.ktext 0x80000180` lee el carácter y regresa con `eret`.

2. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

En el sondeo (*polling*) es la CPU quien consulta repetidamente el registro de estado del dispositivo para saber si hay un dato disponible. Es un esquema sencillo y determinista, pero la CPU permanece ocupada en una espera activa aunque no ocurra ningún evento.

Con interrupciones es el dispositivo quien avisa al procesador cuando requiere atención. La CPU puede dedicarse a otras tareas entretanto y la latencia de respuesta es menor, a cambio de mayor complejidad: se necesita una rutina de servicio, guardar el contexto, habilitar las interrupciones en el dispositivo y en el procesador, y la depuración es más difícil porque la ejecución es asíncrona.

3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

Se elimina el tiempo perdido en esperas activas, de modo que los ciclos de CPU se emplean en trabajo útil. Además, mejora la capacidad de respuesta ante eventos urgentes, se reduce el consumo energético (la CPU puede entrar en reposo hasta que llegue la interrupción) y se hace posible la multiprogramación, ya que mientras un proceso espera a un dispositivo la CPU puede ejecutar otro.

4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

Se emplean registros del coprocesador 0, accesibles con `mfc0` y `mtc0`:

- **Status** ($12): contiene el bit de habilitación global IE (bit 0), el bit EXL (bit 1) que se activa al atender una excepción, la máscara de interrupciones IM (bits 8–15) y el modo kernel/usuario.
- **Cause** ($13): contiene el código de la excepción (ExcCode, bits 2–6) y las interrupciones pendientes (IP, bits 8–15).
- **EPC** ($14): dirección de la instrucción donde se produjo la excepción o interrupción, usada por `eret` para retornar.
- **BadVAddr** ($8): dirección inválida que causó un fallo de acceso.
- **Count** ($9) y **Compare** ($11): temporizador del procesador.

También son relevantes los registros `$k0` y `$k1`, reservados para el kernel, y los registros mapeados en memoria del teclado y la pantalla (`0xFFFF0000`–`0xFFFF000C`).

5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

La interrupción puede ocurrir entre dos instrucciones cualesquiera y el programa interrumpido no sabe que fue interrumpido. Si la rutina modifica registros (por ejemplo `$t0`, `$t1` o `$at`) sin restaurarlos, al regresar el programa continuaría con valores corruptos. Por ello la rutina guarda en memoria los registros que va a utilizar y los restaura justo antes de `eret`. En `ejercicio1_buffer_teclado.asm` se guardan `$at`, `$t0`, `$t1` y `$t2` en la zona `.kdata` (`sv_at`, `sv_t0`, `sv_t1`, `sv_t2`).

6. Momentos en que pueden generarse excepciones en un sistema MIPS32.

a) Enumera al menos 4 situaciones en las que se pueda generar una excepción (por ejemplo: desbordamiento aritmético, fallo de dirección, etc.).

Algunas situaciones son: desbordamiento aritmético en `add`, `addi` o `sub`; dirección no alineada o fuera de rango en `lw`/`sw` o al captar una instrucción; instrucción reservada o ilegal; ejecución de `syscall` o `break`; fallo de TLB o de página; coprocesador no disponible; y las interrupciones externas (teclado, reloj).

b) Explica qué etapas del pipeline pueden provocar una excepción y por qué.

Una excepción se detecta en la etapa donde ya se dispone de la información necesaria: en *IF* (dirección de instrucción inválida o fallo de TLB), en *ID* (instrucción ilegal, `syscall`, `break`), en *EX* (desbordamiento aritmético), y en *MEM* (dirección de datos no alineada o inválida, fallo de TLB). En *WB* normalmente no se producen. Como varias instrucciones están en vuelo a la vez, la excepción debe ser precisa: las instrucciones anteriores completan, y la afectada y las posteriores se descartan.

7. Estrategias de tratamiento de excepciones e interrupciones

a) Explica las diferencias entre interrupciones y excepciones (¿son síncronas o asíncronas?).

Las excepciones son *síncronas*: las provoca la propia instrucción en ejecución y se repiten si se ejecuta de nuevo el mismo código con los mismos datos. Las interrupciones son *asíncronas*: provienen de eventos externos al flujo de instrucciones (teclado, reloj, etc.) y pueden llegar en cualquier momento.

b) Describe brevemente dos estrategias para tratar excepciones en un sistema MIPS32. ¿Cómo se redirige la ejecución hacia la rutina de servicio? ¿Cuál es la función del registro EPC (Exception Program Counter)?

Una estrategia es el *vector único*: el hardware salta siempre a una dirección fija (`0x80000180`) y el software lee `Cause` y despacha según el ExcCode, que es lo que hacen los programas de esta práctica. Otra es el uso de *vectores separados o interrupciones vectorizadas*, donde cada causa o fuente tiene su propia dirección de entrada, evitando el despacho por software.

La ejecución se redirige porque, al aceptar la excepción, el hardware escribe el PC en EPC, registra la causa en Cause, pone EXL en 1 y carga en el PC la dirección del manejador. El registro **EPC** guarda la dirección de la instrucción afectada (o la siguiente, en una interrupción), de modo que `eret` pueda retomar la ejecución, o el manejador pueda calcular dónde reanudar (por ejemplo EPC + 4 para saltarse una `syscall`).

8. Habilitación de interrupciones en dispositivos y procesador

a) Explica cómo se habilitan las interrupciones en: el teclado, la pantalla y el procesador (detalla qué bits del registro Status deben modificarse).

Teclado: se escribe 1 en el bit 1 (Interrupt Enable) del registro `0xFFFF0000` (Receiver Control). *Pantalla*: se escribe 1 en el bit 1 del registro `0xFFFF0008` (Transmitter Control). *Procesador*: en **Status** se debe poner en 1 el bit 0 (IE), asegurar que el bit 1 (EXL) esté en 0, y activar los bits de máscara IM correspondientes (bit 8 para el teclado y bit 9 para la pantalla en MARS). En los programas se hace con:

```
1         li   $t0, RCTRL
2         li   $t1, 2
3         sw   $t1, 0($t0)         # habilita interrupcion del teclado
4         mfc0 $t0, $12
5         ori  $t0, $t0, 0x0101    # IE (bit 0) + IM8 (bit 8)
6         mtc0 $t0, $12
```

b) ¿Qué pasaría si habilitamos interrupciones en los dispositivos, pero no en el procesador?

El dispositivo levantaría su señal, pero la CPU la ignoraría: el evento quedaría pendiente en Cause y no se ejecutaría ninguna rutina de servicio. En los ejercicios, el buffer permanecería vacío y el pulsador nunca se detectaría.

9. Procesamiento de interrupciones

a) Describe paso a paso qué ocurre cuando se produce una interrupción de reloj: desde que el evento ocurre hasta que la rutina de servicio termina; ¿qué registros se guardan y restauran?; ¿qué hace el hardware y qué hace el software (sistema operativo o rutina)?

Los pasos son:

1. El temporizador alcanza el valor de comparación (Count = Compare) y activa su línea de interrupción.
2. La CPU la reconoce al terminar la instrucción en curso, si IE = 1, EXL = 0 y el bit IM está activo.
3. **Hardware**: guarda el PC en EPC, escribe el código en Cause, pone EXL = 1 (bloquea nuevas interrupciones) y salta a `0x80000180`.
4. **Software**: guarda los registros que usará (`$at`, `$t0`, ...), lee Cause para identificar la fuente, atiende al reloj (reprograma Compare, actualiza contadores o decide cambiar de proceso), y restaura los registros.
5. **Hardware**: `eret` limpia EXL y carga el PC con EPC, reanudando el programa.

Por hardware se guardan EPC, Cause y el estado de EXL en Status; por software se guardan y restauran los registros de propósito general que usa la rutina.

b) ¿Por qué es importante guardar el contexto (registros generales, EPC, Status) al entrar en la rutina?

Para no corromper el programa interrumpido y poder reanudarlo exactamente en el mismo estado. Guardar además EPC y Status es imprescindible si se permiten interrupciones anidadas, ya que una nueva interrupción sobrescribiría EPC y se perdería el punto de retorno.

10. Interrupciones de reloj y control de ejecución

a) Explica cómo una interrupción de reloj puede usarse para: evitar que un programa quede en un bucle infinito; finalizar programas que superan un tiempo máximo de ejecución.

El sistema operativo programa el temporizador antes de ceder la CPU a un programa. Si este entra en un bucle infinito, el reloj interrumpe de todas formas, el manejador recupera el control y puede cambiar a otro proceso (planificación expulsiva). Si además se lleva la cuenta del tiempo consumido y este supera un máximo establecido, el manejador puede terminar el programa y liberar sus recursos.

b) ¿Qué debe hacer el software si el programa finaliza antes de que ocurra la interrupción de reloj?

Debe cancelar o desactivar el temporizador (o reprogramarlo) para que no interrumpa fuera de contexto, y no termine por error a otro programa o altere el estado del sistema.

11. Análisis y Discusión de los Resultados

Ejercicio 1: buffer de teclado con filtro de caracteres

En el archivo `ejercicio1_buffer_teclado.asm` se define un buffer circular de 16 bytes con los índices `head` (escritura), `tail` (lectura) y el contador `count`. El programa principal habilita la interrupción en el registro de control del teclado y los bits IE e IM8 de Status. Luego mide los 20 segundos con `syscall` 30 (tiempo del sistema en milisegundos) mientras las interrupciones llegan durante la espera.

La rutina de servicio (`.ktext 0x80000180`) guarda los registros que utiliza, comprueba en Cause que se trata de una interrupción de teclado, lee el carácter de `0xFFFF0004` (lo que limpia el bit *Ready*), lo filtra y lo inserta en el buffer. Si el buffer está lleno se descarta el dato más antiguo. Se añadió además la conversión de minúsculas a mayúsculas, de modo que lo escrito en minúscula también se guarda como letra A–Z; el resto de caracteres se descarta. Al terminar el intervalo se deshabilita IE para vaciar e imprimir el buffer sin condiciones de carrera con la rutina de servicio, se vuelve a habilitar y el proceso se repite indefinidamente.

```
# Lab 4 - Ejercicio 1: buffer circular de teclado con filtro A-Z
# Uso en MARS: Tools > Keyboard and Display MMIO Simulator > "Connect to MIPS",
# luego ensamblar y ejecutar. Escribir en el campo de teclado del simulador.
# El tiempo (20 s) se mide con syscall 30 (tiempo del sistema en ms).

        .eqv RCTRL  0xffff0000
        .eqv RDATA  0xffff0004
        .eqv BUFSZ  16
        .eqv MS20   20000

        .data
buffer:  .space 16
head:    .word 0            # posicion de escritura
tail:    .word 0            # posicion de lectura
count:   .word 0            # elementos validos
msgIni:  .asciiz "\n--- Escriba (solo A-Z se guardan), 20 segundos ---\n"
msgBuf:  .asciiz "\nContenido del buffer: "
nl:      .asciiz "\n"
chbuf:   .space 2

        .kdata
sv_at:   .word 0
sv_t0:   .word 0
sv_t1:   .word 0
sv_t2:   .word 0

        .text
        .globl main
main:
        # habilitar interrupcion del teclado en el dispositivo (bit 1)
        li   $t0, RCTRL
        li   $t1, 2
        sw   $t1, 0($t0)
        # habilitar interrupciones en el procesador: Status IE (bit0) + IM bit 8
        mfc0 $t0, $12
        ori  $t0, $t0, 0x0101
        mtc0 $t0, $12

ciclo:
        la   $a0, msgIni
        li   $v0, 4
        syscall

        li   $v0, 30            # tiempo inicial en $a0 (ms, parte baja)
        syscall
        move $s0, $a0

espera:
        li   $v0, 30
        syscall
        subu $t0, $a0, $s0
```

```
52          blt  $t0, MS20, espera  # las interrupciones llegan durante esta espera
53
54          # --- vaciar e imprimir el buffer (seccion critica: se deshabilita IE) ---
55          mfc0 $t0, $12
56          andi $t0, $t0, 0xfffe
57          mtc0 $t0, $12
58
59          la   $a0, msgBuf
60          li   $v0, 4
61          syscall
62
63  vaciar:
64          lw   $t0, count
65          beqz $t0, finVaciar
66          lw   $t1, tail
67          lb   $a0, buffer($t1)
68          li   $v0, 11
69          syscall
70          addi $t1, $t1, 1
71          li   $t2, BUFSZ
72          rem  $t1, $t1, $t2
73          sw   $t1, tail
74          addi $t0, $t0, -1
75          sw   $t0, count
76          j    vaciar
77  finVaciar:
78          la   $a0, nl
79          li   $v0, 4
80          syscall
81
82          # reactivar interrupciones y repetir
83          mfc0 $t0, $12
84          ori  $t0, $t0, 1
85          mtc0 $t0, $12
86          j    ciclo
87
88  # ================================================================
89  # Rutina de servicio de excepciones / interrupciones
90  # ================================================================
91          .ktext 0x80000180
92          move $k1, $at             # guardar $at (MARS ignora .set noat)
93          sw   $k1, sv_at
94          sw   $t0, sv_t0
95          sw   $t1, sv_t1
96          sw   $t2, sv_t2
97
98          mfc0 $k0, $13             # Cause
99          srl  $t0, $k0, 2
100         andi $t0, $t0, 0x1f       # codigo de excepcion
101         bnez $t0, fin             # no es interrupcion: se ignora
102         andi $t0, $k0, 0x100      # bit 8: interrupcion de teclado
103         beqz $t0, fin
104
105         li   $t0, RDATA
106         lw   $t1, 0($t0)          # leer caracter (limpia Ready)
107         andi $t1, $t1, 0xff
108         blt  $t1, 'a', mayus      # minuscula a-z -> convertir a mayuscula
109         bgt  $t1, 'z', mayus
110         addi $t1, $t1, -32
111 mayus:
112         blt  $t1, 'A', fin        # filtro A-Z
113         bgt  $t1, 'Z', fin
114
115         # insertar en buffer circular (si esta lleno, sobreescribe el mas antiguo)
116         lw   $t0, head
117         sb   $t1, buffer($t0)
118         addi $t0, $t0, 1
119         li   $t2, BUFSZ
120         rem  $t0, $t0, $t2
121         sw   $t0, head
122         lw   $t1, count
123         beq  $t1, BUFSZ, lleno
124         addi $t1, $t1, 1
```

```
        sw   $t1, count
        j    fin
lleno:
        lw   $t1, tail          # descarta el mas antiguo
        addi $t1, $t1, 1
        li   $t2, BUFSZ
        rem  $t1, $t1, $t2
        sw   $t1, tail

fin:
        lw   $t2, sv_t2
        lw   $t1, sv_t1
        lw   $t0, sv_t0
        lw   $k1, sv_at
        move $at, $k1
        mtc0 $zero, $13          # limpiar Cause
        mfc0 $k0, $12
        ori  $k0, $k0, 1         # reactivar IE (EXL se limpia con eret)
        mtc0 $k0, $12
        eret
```

Ejercicio 2: semáforo con pulsador

En `ejercicio2_semaforo.asm` el semáforo es una máquina de estados verde → amarillo → rojo → verde. En verde el programa espera la bandera `pulsado`, que activa la rutina de servicio al recibir la tecla `s`. Después se muestran los mensajes y se espera con un temporizador de 20, 10 y 30 segundos, y el ciclo se repite. Mientras se espera con el temporizador, la tecla `s` se ignora.

También se implementó la tecla `e`: la rutina de servicio activa la bandera `salir`, que el programa consulta en la espera del pulsador y dentro del temporizador, e imprime un mensaje y finaliza con `syscall` 10. Durante las pruebas se detectó que el temporizador terminaba de inmediato porque la duración se guardaba en `$a1`, y `syscall` 30 sobrescribe `$a0` y `$a1` con el tiempo; la solución fue guardar la duración en `$s1`.

```
# Lab 4 - Ejercicio 2: semaforo con pulsador (tecla 's')
# Uso en MARS: Tools > Keyboard and Display MMIO Simulator > "Connect to MIPS".
# La tecla genera interrupcion; los temporizadores se miden con syscall 30 (ms).

        .eqv RCTRL  0xffff0000
        .eqv RDATA  0xffff0004

        .data
pulsado: .word 0
salir:   .word 0
msgE:    .asciiz "Tecla 'e' pulsada: fin de la ejecucion\n"
msgV:    .asciiz "Semaforo en verde, esperando pulsador\n"
msgP:    .asciiz "Pulsador activado: en 20 segundos, el semaforo cambiara a amarillo\n
    "
msgA:    .asciiz "Semaforo en amarillo, en 10 segundos, semaforo en rojo\n"
msgR:    .asciiz "Semaforo en rojo, en 30 segundos, semaforo en verde\n"

        .kdata
sv_t0:   .word 0
sv_t1:   .word 0

        .text
        .globl main
main:
        li   $t0, RCTRL
        li   $t1, 2
        sw   $t1, 0($t0)         # habilita interrupcion del teclado
        mfc0 $t0, $12
        ori  $t0, $t0, 0x0101    # IE + IM8
        mtc0 $t0, $12

ciclo:
        sw   $zero, pulsado
        la   $a0, msgV
        li   $v0, 4
```

```
35          syscall
36  esperaPulsador:
37          lw   $t0, salir
38          bnez $t0, terminar
39          lw   $t0, pulsado
40          beqz $t0, esperaPulsador
41
42          la   $a0, msgP
43          li   $v0, 4
44          syscall
45          li   $s1, 20000
46          jal  temporizador
47
48          la   $a0, msgA
49          li   $v0, 4
50          syscall
51          li   $s1, 10000
52          jal  temporizador
53
54          la   $a0, msgR
55          li   $v0, 4
56          syscall
57          li   $s1, 30000
58          jal  temporizador
59
60          j    ciclo
61
62  terminar:
63          la   $a0, msgE
64          li   $v0, 4
65          syscall
66          li   $v0, 10
67          syscall
68
69  # temporizador: espera $s1 milisegundos (syscall 30 pisa $a0 y $a1,
70  # por eso la duracion se guarda en $s1). Si se pulsa 'e', termina el programa.
71  temporizador:
72          li   $v0, 30
73          syscall
74          move $t8, $a0
75  tloop:
76          lw   $t0, salir
77          bnez $t0, terminar
78          li   $v0, 30
79          syscall
80          subu $t9, $a0, $t8
81          bltu $t9, $s1, tloop
82          jr   $ra
83
84  # ================================================================
85          .ktext 0x80000180
86          move $k1, $at
87          sw   $t0, sv_t0
88          sw   $t1, sv_t1
89          mfc0 $k0, $13
90          srl  $t0, $k0, 2
91          andi $t0, $t0, 0x1f
92          bnez $t0, fin
93          andi $t0, $k0, 0x100
94          beqz $t0, fin
95          li   $t0, RDATA
96          lw   $t1, 0($t0)          # lee caracter
97          andi $t1, $t1, 0xff
98          beq  $t1, 'e', pedirSalir
99          bne  $t1, 's', fin
100         li   $t1, 1
101         sw   $t1, pulsado         # marca pulsador (el ciclo principal lo consume)
102         j    fin
103 pedirSalir:
104         li   $t1, 1
105         sw   $t1, salir           # el programa principal termina
106 fin:
107         lw   $t1, sv_t1
```

```
108         lw   $t0, sv_t0
109         move $at, $k1
110         mtc0 $zero, $13
111         mfc0 $k0, $12
112         ori  $k0, $k0, 1
113         mtc0 $k0, $12
114         eret
```

Discusión general

Ambos ejercicios muestran que las interrupciones permiten atender un dispositivo sin bloquear el programa principal: la tecla se procesa en cuanto llega, aunque el programa esté en medio de una espera. Un límite importante es que MARS no ofrece una interrupción de temporizador estándar, por lo que el tiempo se mide con una espera activa sobre `syscall` 30 y sólo el teclado usa interrupciones reales. En hardware real se programaría el registro `Compare` de CP0 para obtener la interrupción de reloj y evitar esa espera activa. Para que el programa reciba las teclas es necesario usar el *Keyboard and Display MMIO Simulator* con *Connect to MIPS*, ya que la lectura por consola mediante `syscall` bloquea y no genera interrupciones.